

Public Traceability in Threshold Decryption

Sébastien Canard, Nathan Papon, Duong Hieu Phan^[0000–0003–1136–4064]

Télécom Paris, LTCI, Institut Polytechnique de Paris
`{sebastien.canard,nathan.papon,hieu.phan}@telecom-paris.fr`

Keywords: Threshold decryption · Traitor Tracing · IPP codes

Abstract. Tracing techniques have been used to identify users who have leaked their decryption keys in a secure multi-receiver encryption system. Very recently, in the field of distributed cryptography, where trust is distributed, Boneh et al. extended traitor tracing to the framework of threshold decryption, where a single user doesn't hold the whole secret to decrypt but needs to collaborate with others. However, the tracing capacity in their collusion-secure codes-based schemes is still centralized: only the authority holding the secret tracing key can perform tracing. We continue in the direction of not relying on a single entity and propose decentralizing tracing in this context so that the tracing procedure does not need to rely on any secret key and can be done by anyone. Technically, as binary collusion-secure codes only support secret tracing, we switch to robust q -ary IPP codes supporting public tracing. This requires us to generalize the bipartite threshold KEM for two users in Boneh et al.'s paper to q -partite KEM for q users. In terms of security, their static one-sided security in the binary case is not appropriate, which requires us to define an adaptive one-sided security notion for q -partite KEM to be compatible with q -ary IPP codes. Finally, we generalize the Boneh et al. construction to achieve this security notion and achieve public traceability for threshold decryption without degrading efficiency.

1 Introduction

In content distribution, a broadcaster delivers messages to a large audience such as in the case of paid television services. However, as real-life experience has shown, unauthorized users (or pirates) often find ways to bypass subscription fees and access the content for free. These malicious actors can create decryption devices, known as decoders, which enable non-subscribers to decrypt and view the content illicitly.

To resolve this issue, the concept of traitor tracing was introduced in [11]. It aims to identify the source of such illegal decoders while ensuring strong security guarantees for honest users—for example, by preventing false accusations.

More recently, the question of traceability and accountability has arisen in the context of decentralised decryption. For instance, in decentralised electronic voting, if the adversary attempts to corrupt more parties than the threshold, then nothing stops them from running decryptions. Being able to trace users who sell

their keys or accept bribes from an adversary would deter them to misbehave before the vote is finalised. Another scenario, envisioned by [5] considers MEV extraction in encrypted mempools, where a decoder could help decoding the mempool before the last block is added on chain, therefore making the encryption useless. This has motivated a new interest into extending traitor tracing schemes for decentralised systems. We refer the reader to [5] for additional motivation on practical applications.

Traitor Tracing [11] is formalised as a multi-receiver encryption scheme in which coalitions of malicious users collaborating to build pirate decryption devices can be identified by a tracing algorithm. With possession of such a pirate decryption device, a tracer can recover at least one member of the malicious coalition. The most common tracing model is when the tracer interacts with the pirate decoder in a *black-box* manner. However, there are many other models, such as *non-black-box*, *single-key black-box tracing* or *black-box confirmation* [3]. Also, the decoder can be *stateless*, meaning that it can be reset by the tracer to its default setting. It is opposed to *stateful* decoders [16] in which they keep track of previously seen queries. Finally, if the coalition’s size can not exceed a maximum number of traitors, the scheme is said to be in the *bounded model*, which is more restrictive than the *unbounded model* but in practice leads to more efficient schemes.

Private vs Public traceability. Traitor Tracing schemes have seen many improvements over the years. An interesting line of work focuses on the authority yielded by the tracer. Schemes with *private tracing* requires special information that can only be held by a trusted authority in charge of the tracing. Concrete scenarios tend to reduce the trust placed on such a tracer. The strongest notion one can achieve is non repudiation [22]. However such schemes require interaction with the users at setup, such as 2-party computations [22], commitment protocols [23] or oblivious polynomial evaluation [25,18,15]. Without interaction, it is possible to achieve a weaker property, named *public traceability*, put forth in [10], with the possibility to run the tracing procedure using only public information. This allows anyone to run tracing without the need for any secret tracing key held by a central authority. Another significant advantage of public traceability is the possibility to delegate the tracing procedure. Indeed, a bottleneck in traitor tracing is the high tracing complexity, and it would not be practical if only a central authority can do that. This level of security has been achieved in many schemes in the classical model [8,9,10,24].

Traitor Tracing for Threshold Decryption. The recent work of [5] opens an exciting new line of study, introducing a traitor tracing with a threshold of users for the decryption. Threshold decryption schemes aim to distribute the capability of decrypting ciphertexts among all or some of the users. Upon decryption of a ciphertext, a user does not get the original plaintext but only a share which needs to be combined with the ones obtained by other users. A key feature of such schemes is that a group smaller than the threshold cannot learn anything about the original plaintext; their shares are useless on their own and do not

reveal any information. Adapting traitor tracing to such a threshold setting is not trivial. In the degenerated case where the threshold is set to one, a threshold traitor tracing scheme is nothing but a classical traitor tracing scheme. Hence, the design of such schemes seems to be more challenging and classical traitor tracing methods may not extend to this setting easily.

As considered in [5], the most serious situation arises when a coalition of traitors is larger than the threshold. A practical scenario could involve these traitors colluding to design a pirate decoder, well obfuscated, capable of decrypting and combining ciphertext shares to output a plaintext. Since the decoder is supposed to be well obfuscated, the only way to extract information from it is via black-box access with ciphertext queries and observing what it decrypts. Some works deal with stateful decoders but the resulting schemes are all rather inefficient [16]. Hence most works, including ours, consider a stateless decoder.

A shortcoming of the solution presented in [5] is the lack of public traceability. The tracer needs a private tracing key to run the procedure. In alignment with the philosophy of threshold cryptography and its promise of decentralization, our main motivation is to fill this gap by achieving public traceability in the context of threshold decryption.

Bipartite Encryption. The solution of [5] relies on the use of two cryptographic primitives. They combine collusion secure codes [7] with a new primitive that they called *Bipartite Encryption* (BTKEM for short). In such a primitive, each user receives ℓ pairs of secret keys. Each pair is seen as a *position* having a *left* and a *right* secret key. Similarly, ciphertexts are encrypted for a given *position* and are composed of a *left* and a *right* side. To decrypt, a user will use one of his *left* key with the *left* side of a ciphertext (resp. *right* key with *right* side).

The central requirement for the correctness of the scheme is the *two-sided correctness*. Loosely speaking, the decryption must output the same share, no matter which side of the ciphertext was used. The combination must be correct according to classical correctness definition of threshold scheme, it doesn't matter which side the shares come from.

On top of the classical semantic security required from threshold schemes, a bipartite encryption scheme is required to have *one-sided security*. At a very high level, an adversary in possession of only one side of secret keys, say *left*, cannot learn any meaningful information from the *right* side of a ciphertext. This definition is central for the tracing procedure to work. When combining a BTKEM with collusion secure codes, the tracing consists in sending to the private device a modified ciphertext. Users will receive only one key per position, depending on their personal codewords. As an example, say user i 's codeword is $\omega_i = 010$. Then i will receive the set of keys $(sk_{1,0}, sk_{2,1}, sk_{3,0})$. For position 1, i will only be able to decrypt the *left* side of a ciphertext. By the *one-sided security* property for the first position, i will not notice if the *right* side of a ciphertext is a valid encryption or if it has been modified. Now, suppose i is a traitor and his first key is used by the decoder. If the tracer switches for the first position the left side of a ciphertext and the decoder notices it, the tracer will accuse the device of having a *left* key. By repeating this operation for every position, the tracer

constructs a pirate codeword, then uses the tracing procedure of the collusion secure code. But these codes require a private key to trace a user. Hence, if one desires to obtain a public tracing algorithm, one needs to use another kind of code. In this paper, we propose IPP codes and therefore adapt the bipartite encryption to combine it with a code defined on a larger alphabet.

1.1 Our Contribution

To reach public tracing, our main result is then the generalisation of the Bipartite Encryption scheme from [5] to the case of q -Partite. If the two-sided correctness is easily transposable to the q -partite case, this is not the case for the one-sided security. In particular, we generalize this definition to an *adaptive* version of such a one-side security, and prove it secure under the standard DDH assumption. We then combine our new q -Partite Encryption scheme with robust IPP codes [2] to construct a traitor tracing decryption scheme with public traceability for a coalition which size is greater than the threshold.

1.2 Technical Overview

Bipartite threshold key encapsulation message. In order to understand how a q -Partite TKEM scheme works, it is important to see the intuition behind the bipartite case. In such a scheme, the key generation algorithm samples pairs of secret keys and, as said above, each user i ($i \in [n]$) receives ℓ pairs of keys (the *left* one and the *right* one), where ℓ is a parameter of the system. The set of $2n\ell$ secret keys can be represented with the following matrix:

$$\begin{pmatrix} sk_{1,0}^1 & sk_{1,1}^1 & \dots & sk_{i,0}^1 & sk_{i,1}^1 & \dots & sk_{n,0}^1 & sk_{n,1}^1 \\ \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots \\ sk_{1,0}^j & sk_{1,1}^j & \dots & sk_{i,0}^j & sk_{i,1}^j & \dots & sk_{n,0}^j & sk_{n,1}^j \\ \vdots & \vdots & & \vdots & \vdots & & \vdots & \vdots \\ sk_{1,0}^\ell & sk_{1,1}^\ell & \dots & sk_{i,0}^\ell & sk_{i,1}^\ell & \dots & sk_{n,0}^\ell & sk_{n,1}^\ell \end{pmatrix},$$

where the rows of the matrix are called *positions*.

The encryption process necessitates to specify a position $j \in [\ell]$, and eventually outputs a ciphertext with two sides: *left* and *right*. To decrypt, one uses the *left* (resp. *right*) key with the *left* (resp. *right*) side of the ciphertext for the given position. With enough shares of a ciphertext, a combiner can use all those contributions to recover the original plaintext, using the combining algorithm of the underlying secret sharing scheme. As said above, three security properties are requested for such a primitive: two-sided correctness, traditional semantic security, and lastly one-sided security.

The need for a q -partite TKEM. At a very high level, a bipartite KEM scheme only needs *two* sides because when it is used in the traitor tracing scheme, it is combined with collusion secure codes which ones are defined on a binary alphabet. As a consequence, in the traitor tracing scheme for threshold decryption of Boneh et al. [5], each user is assigned a personal codeword and receives ℓ keys, depending on his/her codeword. More precisely, for position j , user i receives $sk_{i,0}^j$ (left key) if the letter at position j in his personal codeword is 0, and $sk_{i,1}^j$ (right key) if the latter is 1. Hence, during decryption, user i will only be able to use the side of the ciphertext for which he has the corresponding secret key.

The problem is that the generation of such codewords necessitates the knowledge of a secret information, so that the tracing procedure is necessarily private. In order to achieve public tracing, we need to find an alternative. Indeed, q -ary codes with Identifiable Parent Property (IPP codes for short) is a relevant candidate, as it was combined with a q -users multi-receivers system in [24]. These codes are nevertheless defined on a larger alphabet, namely of size q . Such a code provides a public tracing algorithm and have already been used to design traitor tracing scheme with public traceability. But in order to use q -ary IPP codes, the notion of bipartite KEM is not enough and we need to add more parts to the ciphertexts and give more keys to users. We call such a new primitive a q -Partite TKEM scheme. Our purpose in this paper is to properly define this primitive, to propose an efficient construction, and eventually to show how to use it to design a traitor tracing threshold encryption scheme with public traceability.

Description of a q -partite TKEM. In a q -Partite TKEM scheme, each user receives $q\ell$ secret keys where q and ℓ are parameters of the system. The set of secret keys can be represented by the following matrix:

$$\begin{pmatrix} sk_{1,1}^1 \dots sk_{1,q}^1 \dots sk_{i,1}^1 \dots sk_{i,q}^1 \dots sk_{n,1}^1 \dots sk_{n,q}^1 \\ \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \\ sk_{1,1}^j \dots sk_{1,q}^j \dots sk_{i,1}^j \dots sk_{i,q}^j \dots sk_{n,1}^j \dots sk_{n,q}^j \\ \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \\ sk_{1,1}^\ell \dots sk_{1,q}^\ell \dots sk_{i,1}^\ell \dots sk_{i,q}^\ell \dots sk_{n,1}^\ell \dots sk_{n,q}^\ell \end{pmatrix}$$

Again, the rows are called *positions* and we call the columns *letters*. Each user is associated $q\ell$ letters, for a total of $qn\ell$ secret keys. The encryption algorithm, given a position $j \in [\ell]$, outputs a ciphertext with q parts. To decrypt, one has to choose one of his/her q secret keys for position j , and use it with the corresponding part of the ciphertext. The one to one correspondence is shown

in the following matrix, for user i and a message encrypted for position j :

$$\begin{pmatrix} sk_{i,1}^j & c_1 \\ \vdots & \\ sk_{i,\alpha}^j & c_\alpha \\ \vdots & \\ sk_{i,q}^j & c_q \end{pmatrix}$$

For example, a user could use the letter $\alpha \in [q]$, and in this case needs to use $sk_{i,\alpha}^j$ and c_α together.

Distinction between TKEM, BTKEM and QTKEM. A Threshold Key Encapsulation Mechanism (Threshold KEM or TKEM for short) presented in section 2.1 is the standard threshold variant of a Key Encapsulation Mechanism (KEM) scheme. The encryption algorithm takes as input only a public key, and outputs a session key k and a ciphertext c . The correctness requirement is that given enough decryption shares d_i , a combiner can recover the original session key. Semantic security as defined in definition 3.2 is the minimal requirement for the security of such schemes. Both Bipartite KEM and qPartite KEM are Threshold KEMs but with additional properties. First, the Encrypt algorithm now takes as input the public key and an additional index which is used to select the subset of secret key that could be used to decrypt. In the Bipartite (resp qPartite) KEM the algorithm outputs a session key with a ciphertext divided into 2 (resp q) parts. Following this distinction, the correctness of the schemes (both Bipartite and qPartite KEM) need to be modified accordingly. More details will be given in section 3.2. Finally, the Bipartite KEM and qPartite KEM have an additional security property called *one-sided security* for Bipartite KEM and *adaptive one-sided security* for qPartite KEM. The security of the tracing heavily relies on this property in the traitor tracing schemes that combines Bipartite KEM with robust collusion secure codes and qPartite KEM with robust q -ary IPP codes.

Efficiency comparison. Compared to the bipartite KEM in [5], we have to increase the number of private and public keys by a ratio of $\lfloor q/2 \rfloor$. This is exactly the ratio between our alphabet size and theirs. Since the bipartite KEM is combined with a binary code, they only need pairs of private keys ($2n\ell$ in total) and triplets of public keys (3ℓ in total). In our scheme, since our QTKEM is later on combined with a code of alphabet size q , we increase the number of secret keys to $qn\ell$ as detailed in the previous section and our number of public keys expands to $(q+1)\ell$.

In more details, [5] rely on robust collusion secure codes by Boneh, Kiayias and Montgomery [4]. This binary code has codewords length $\ell = O(f^2 \log n^2 \log(n/\kappa)/(1-\theta))$ where n is the number of users, f is the maximal number of traitors, κ the error probability in identifying traitors and θ the maximal proportion of letters deviating from the marking assumption (see section 2 for more details).

In this work, we use robust IPP codes, where two constructions were given in [12]. The first one has codewords length $\ell = O(f^2 \log n / (1 - \theta - f\theta))$ and alphabet size $q = \Theta(f^2 / (1 - \theta - f\theta))$ but to ensure their code is not a random code, they apply the Porat-Rothschild derandomization procedure which results in a decoding algorithm exponential in the dimension of the code. The second construction that we will use, based on Reed-Solomon codes is more practical and achieves codewords length $\ell = O(\frac{2d \log n}{\log(d \log n)})$ where $d = f^2 / (1 - \theta - f\theta)$ and alphabet size $q = \ell$.

In brief, if robust IPP codes have slightly shorter codewords, they are defined on significantly larger alphabets. Our scheme suffers from the increase of a factor $q/2$ in the number of keys and ciphertexts component a factor $\ell/2$ when using the second construction from [12]. However, the efficiency of our scheme and [5] is still comparable since the codewords length is smaller with robust IPP codes. Note that contrary to robust collusion secure codes, robust IPP codes have a zero probability of errors in identifying traitors. This is summed up in Table 1.

Following that, the complexity of the **KeyGen** and **Enc** see their complexity

Schemes	$ \text{sk} $	$ \text{pk} $	$ c $
Naive	$2n\ell sk_{TKEM} $	$2n\ell pk_{TKEM} $	$O(n)$
BTkEM	$2n\ell \mathbb{G}$	$3\ell \mathbb{G}$	$2\mathbb{G}$
BTBF	$2n\ell \mathbb{G}$	$3\mathbb{G}_1$	$2\mathbb{G}_1 + 2\mathbb{G}_2$
This work	$qn\ell \mathbb{G}$	$(q+1)\ell \mathbb{G}$	$q\mathbb{G}$

Table 1. Number of private keys, public keys and ciphertexts parts where n is the number of users, ℓ is a parameter common for both constructions and q a parameter specific to our construction. $\mathbb{G}$ is a group of prime order p and $\mathbb{G}_1, \mathbb{G}_2$ are two pairing friendly cyclic groups of order p . $|sk_{TKEM}|$ and $|pk_{TKEM}|$ are the sizes of the secret and public key of the underlying threshold KEM used to instantiate the naive construction of the BTkEM.

roughly grow by a factor q in our QTKEM compared to [5], because of the new alphabet size parameter. However, the **Dec** and **Combine** algorithms have the same complexity since they do not rely on it. In **Dec**, a user simply chooses a key/component pair and computes his decryption share, and **Combine** algorithm is simply the reconstruction phase of the Shamir Secret Sharing. Note that the complexities of the four algorithms **KeyGen**, **Enc**, **Dec**, **Combine** are the same for the QTKEM and the Traitor Tracing scheme. Finally, our **Trace** algorithm from the traitor tracing scheme has a complexity slightly greater than [5], because of the alphabet size q and the decoder's advantage ε . Hence, our tracing algorithm sees its complexity grow by roughly a factor $qn^2/\lambda\varepsilon^2$. Table 2 shows the difference in complexities of the algorithms. In [5], they propose a second construction of their BTkEM with shorter public keys and relies on pairings. The construction is proven secure under the Augmented Decisional Diffie-Hellman assumption and in the Random Oracle model. Details on this second construction will not be discussed in this paper since our construction is based on the standard Deci-

sionnal Diffie-Hellman assumption, hence we compare it to the first construction of the BTKEM.

Schemes	Enc	Dec	Combine	Trace	False Positive	Tracing
TT from [5]	$O(1)$	$O(1)$	$O(t^2)$	$O(3\ell\lambda^2)$	✓	Private
This work	$O(q)$	$O(1)$	$O(t^2)$	$O(\frac{8q\ell\lambda n^2}{\epsilon^2})$	✗	Public

Table 2. Algorithms complexity. False Positive refers to the non-zero probability of accusing an innocent user at the end of the tracing procedure.

Notion of adaptive one-sided security. We then need to generalize the security properties of a BTKEM to our generalization. The semantic security works similarly. Regarding the two-sided correctness, we need to define its natural equivalent, called *all-sided correctness*: during the combination algorithm, as long as the combiner is in possession of enough decryption shares, he must be able to recover the plaintext no matter from which part of the ciphertext the decryption shares were obtained.

When it comes to adapting the one-sided security property, it is not so direct. If we keep one-sided security for a q -partite system, it is clearly not enough. Indeed, it is sufficient for the pirate to corrupt two users to break the tracing. Several approaches are possible. One may consider to let the adversary, for some position j , receives only one key out of q . It is also possible to consider an adversary that has $q - 1$ of them. In both cases, it is possible to design a scheme which security can be proved under the DDH assumption (as it will be the case for our construction). However, when it comes to use it for traitor tracing, which is our main objective, these formulations of one-sided security with an almost completely well formed challenge ciphertext lead to an insecure tracing procedure.

Before explaining why, we need to give some details about the tracing procedure for the decryption of a KEM. Indeed, as most tracing strategy does in the literature, the tracer modifies ciphertext steps by steps in order to identify one traitor. More precisely, Boneh et al. [5] use the real-or-random property of their BTKEM. Hence, the decoder receives a session key and a ciphertext and has to distinguish whether the ciphertext is in fact an encryption of the key or a random value. If a decoder distinguishes well, it means that he has a key for the said part and hence has one of the associated secret keys: a traitor has been found.

Coming back to the one-sided security, as the alphabet size q is generally large, it is unlikely for a coalition of at least 2 traitors to have the exact same letter for a given position. If at some position all the traitors have the same letter, the said position is called *undetectable* by property of an IPP code. More details on IPP codes properties are given in section 2. On the other hand, an

adversary in possession of $q - 1$ keys trivially breaks our tracing procedure, as it will be detailed in Section 3.

This is why we propose a more general method, in which we let the adversary choose, after the key generation procedure, the position $j \in [\ell]$ and index $\alpha \in [q]$ for which he wishes to be tested on, in an adaptive way. After having received for the used position j all the keys except the ones related to letter α , he has to guess whether the α -th part of the ciphertext is a proper encryption of the challenge message or a random element, knowing that the first $\alpha - 1$ parts are correct the last $q - \alpha$ parts are random. The adaptive nature of our new security definition leads to the new notion of adaptive one-sided security.

It is important to note that the tracing procedure from the traitor tracing scheme makes heavy use of this security property.

Public traceability in threshold decryption. Equipped with our new tool, we are now able to use it to design our public traceability in threshold decryption. But we need an extra trick for that. Indeed, the tracer in [5] produces three kinds of ciphertext: (i) both parts are related to the session key, (ii) none of them is and (iii) only one of the two is. It then compares the probability for the decoder to correctly guess between each ciphertexts type, as explained above. If we consider now a ciphertext with q parts, as we have with our q -partite KEM, it would take an exponential number of types of ciphertexts to make all the two by two comparisons. To prevent such inefficiency, we use a general linear tracing strategy, adapted from [12] to create a pirate codeword. More details are given in Section 4.

2 Preliminaries

2.1 Threshold Decryption KEM

We begin by recalling the standard definition of a Threshold Decryption Key Encapsulation Mechanism (TKEM). As in [5], we could also consider the case of public key encryption but for convenience, we focus on KEM. A KEM is an encryption scheme where the encryption algorithm only takes as input the public key and outputs a session key along with a ciphertext equal to the encryption of the key. The formal definition is then defined as below:

Definition 1. *A Threshold Decryption KEM is a tuple of four polynomial time algorithms $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Combine})$ defined as follows:*

- $\text{KeyGen}(1^\lambda, n, t)$: *Takes as input the security parameter λ , the number of users and the threshold t . The algorithm outputs the public key pk , the public combiner key pkc and the users secret keys $sk_1, \dots, sk_n$.*
- $\text{Enc}(pk)$: *Takes as input the public key pk . The algorithm outputs a session key k and a ciphertext c .*

- $\text{Dec}(sk_i, c)$: Takes as input the user secret key sk_i and a ciphertext c . The algorithm outputs a decryption share d_i .
- $\text{Combine}(pkc, c, \mathcal{J}, \{d_i\}_{i \in \mathcal{J}})$: Takes as input the public combiner key pkc , a ciphertext c , a set of users $\mathcal{J} \subseteq [n]$ and its associated set of shares $\{d_i\}_{i \in \mathcal{J}}$. The algorithm outputs the session key k or a rejection symbol $\perp$.

A Threshold Decryption KEM is required to satisfy the definitions of correctness and CPA security.

Definition 2 (Correctness).

With overwhelming probability over the randomness of the algorithms, we have for $\lambda \in \mathbb{N}$, $n = n(\lambda)$, $0 < t \leq n$:

$$\forall \mathcal{J} \subseteq [n], \text{ s.t. } |\mathcal{J}| = t, \text{ Combine}(pkc, c, \mathcal{J}, \{d_i\}_{i \in \mathcal{J}}) = k$$

where $(k, c) = \text{Enc}(pk)$, and $\forall i \in \mathcal{J}, d_i = \text{Dec}(sk_i, c)$.

Security. We can define the semantic security definition as a real-or-random game between a challenger and an adversary. The challenger generates a ciphertext and key using the encryption algorithm and samples uniformly at random a key from the key space. It then sends the ciphertext with one of the two keys. The adversary wins the game if he is able to distinguish between a real key encrypted in the ciphertext or a random one. The semantic security is defined in the real-or-random experiment between an adversary and a challenger in figure 1.

1. The adversary outputs a number of parties n and a threshold t , given a security parameter λ .
2. The challenger runs $\text{Keygen}(1^\lambda, n, t)$ to generate $(pk, pkc, sk_1, \dots, sk_n)$.
3. At any time of the game, the adversary can corrupt party $i \in [n]$ and receives sk_i from the challenger. If the adversary corrupts more than t parties, then the challenger stops the game.
4. The challenger samples $b \leftarrow \{0, 1\}$, $(k^{(0)}, c) \leftarrow \text{Enc}(pk)$ and $k^{(1)} \leftarrow \mathcal{K}$ and sends $k^{(b)}, c$ to the adversary.
5. The adversary can eventually corrupt more parties. The challenger also stops the game if the total number of corrupted parties exceeds t .
6. Finally, the adversary returns its guess $b' \in \{0, 1\}$. The adversary wins this game if $b = b'$.

Fig. 1. CPA game for Threshold Decryption KEM and adversary $\mathcal{A}$

We define $\text{Succ}^{\text{CPA}}(\mathcal{A}) = \Pr[b' = b]$ as the probability that the adversary $\mathcal{A}$ wins the game in figure 1.

Definition 3. We say that a TKEM is CPA secure if all probabilistic polynomial time algorithms $\mathcal{A}$ have at most negligible advantage in the game in figure 1,

where $\mathcal{A}$'s advantage is defined as $\text{Adv}_{\text{TKEM}}^{\text{CPA}}(\mathcal{A}) = \left| \text{Succ}^{\text{CPA}}(\mathcal{A}) - \frac{1}{2} \right|$.

2.2 Bipartite Encryption scheme

In this section, we recall the definition of the BTKEM introduced in [5]. As mentioned in the introduction in section 1.2, a BTKEM deviates from a TKEM in its specification of the encryption algorithm and therefore in its correctness as well as the additional security property called *one-sided security* (OSS). The formal definition is given as bellow:

Definition 4. A Bipartite KEM is a tuple of four polynomial time algorithms $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Combine})$ defined as follows:

- $\text{KeyGen}(1^\lambda, n, t, \ell)$: Takes as input the security parameter λ , the number of users n , the threshold t and a number ℓ of positions. It outputs a public key pk , a public combiner key and the set of secret keys $((sk_{1,0}^j, sk_{1,1}^j), \dots, (sk_{n,0}^j, sk_{n,1}^j))_{j \in [\ell]}$.
- $\text{Enc}(pk, j)$: Takes as input the public key pk and an a position $j \in [\ell]$. Outputs a session key k and a ciphertext $c = (c_0, c_1)$ where c_0 is called the left ciphertext and c_1 the right ciphertext.
- $\text{Dec}(j, sk_{i,b}^j, c)$: Takes as input a position j , a secret key $sk_{i,b}^j$, where $b \in \{0, 1\}$, and a ciphertext $c = (c_0, c_1)$. It outputs a decryption share d_i .
- $\text{Combine}(pkc, c = (c_0, c_1), j, \mathcal{J}, \{d_i\}_{i \in \mathcal{J}})$: Takes as input the public combiner key pkc , a ciphertext $c = (c_0, c_1)$, a position j , a set of parties $\mathcal{J} \subseteq [n]$ and a set of decryption shares $\{d_i\}_{i \in \mathcal{J}}$. It outputs a session key k or an error symbol $\perp$.

All-sided correctness. We recall the all-sided correctness of the BTKEM. The correctness of the scheme should hold no matter which parts (*left* or *right*) were used by each user to compute their decryption share.

Definition 5 (Correctness). For all integer $n, t \leq n, \ell, \mathcal{J} \subseteq [n]$ of size t , $\mathcal{E}$ is said to be correct if:

$$\text{Combine}(pkc, c, j, \mathcal{J}, \{d_i\}_{i \in \mathcal{J}}) = k,$$

where $\forall j \in [\ell], (k, c = (c_0, c_1)) = \text{Enc}(pk, j)$ and $\forall i \in \mathcal{J}, b \in \{0, 1\}, d_i = \text{Dec}(j, sk_{i,b}^j, c)$.

Semantic Security. Next, we recall the CPA security property of the BTKEM. It is captured in figure 2. This definition closely resembles that of a standard TKEM.

Definition 6. We say that $\mathcal{E}$ is CPA secure if all probabilistic polynomial time algorithm $\mathcal{A}$ has at most negligible advantage in the game in figure 2, where $\mathcal{A}$'s advantage is defined as $\text{Adv}_{\mathcal{E}}^{\text{CPA}}(\mathcal{A}) = \left| \text{Succ}^{\text{CPA}}(\mathcal{A}) - \frac{1}{2} \right|$ and $\text{Succ}^{\text{CPA}}(\mathcal{A}) = \Pr[b = b']$ is the probability that the adversary outputs the correct bit in the game in figure 2

1. The adversary outputs a number of parties n and a threshold t given a security parameter λ .
2. The Challenger runs $\text{KeyGen}(1^\lambda, n, t, \ell)$ to generate $(pk, pkc, sk_1, \dots, sk_n)$.
3. At any time of the game, the adversary can corrupt party $i \in [n]$ and receives sk_i from the challenger. If the adversary corrupts more than t parties, then the challenger stops the game.
4. The challenger samples a position $u \in [\ell]$, a bit $b \in \{0, 1\}$, generates a ciphertext and key pair $(k^{(0)}, c) \leftarrow \text{Enc}(pk, u)$ and samples a random key $k^{(1)} \leftarrow \mathcal{K}$. Finally, the challenger sends $k^{(b)}, c$ to the adversary.
5. The adversary can eventually corrupt more parties. The challenger stops the game if the total number of corrupted parties exceeds t .
6. Finally, the adversary returns its guess $b' \in \{0, 1\}$. It wins this game if $b = b'$.

Fig. 2. CPA game for BTKEM $\mathcal{E}$ and adversary $\mathcal{A}$

One-sided security. Finally, we recall the definition for one-sided security from [5]. This security property significantly differs from our *adaptive one-sided security* property for QTKEM. In the original *OSS* game, the adversary chooses the position u and side d *before* receiving any public keys, and is asked to distinguish between three possible types of ciphertext. First, if the bit b chosen by the challenger is equal to 0, receives a mixture with a *good* left part (meaning, c_0 can be decrypted into shares of the session key, whereas c_1 is an encryption of uniformly random key. Secondly, if $b = 1$ and $d = 0$ the adversary receives a well formed ciphertext. Lastly, if $b = 1$ and $d = 1$ the adversary receives a uniformly random ciphertext (neither c_0 nor c_1 can decrypt into shares of the session key of the challenge. More details on the difference between the *one side security* and *adaptive one side security* are given in section 3.

Definition 7. We say that a BTKEM $\mathcal{E}$ is *adaptively one-sided secure* if all probabilistic polynomial time algorithm $\mathcal{A}$ have at most negligible advantage in the game in figure 3, where $\mathcal{A}$'s advantage is defined as $\text{Adv}_{\mathcal{E}}^{\text{OSS}}(\mathcal{A}) = \left| \text{Succ}^{\text{OSS}}(\mathcal{A}) - \frac{1}{2} \right|$ and $\text{Succ}^{\text{OSS}}(\mathcal{A}) = \Pr[b = b']$ is the probability that the adversary outputs the correct bit in the game in figure 3.

2.3 Traitor Tracing for Threshold Decryption

We now give a definition of a traitor tracing for threshold decryption scheme. In such a setting, we consider that some users could be traitors that commonly create and publish a pirate decoder, capable of outputting decryption shares. Their purpose is then that nobody should be able to link them to such a public pirate decoder.

We focus on the setting where the coalition of traitors is greater than the threshold. It means that such a coalition can decrypt ciphertexts without the help of any other honest party. Our purpose is to identify at least one of the traitors,

1. The adversary, given a security parameter λ , outputs a number of parties n a threshold t , a number of positions ℓ , a position $u \in [\ell]$ and a side $d \in \{0, 1\}$ and sends them to the challenger.
2. The challenger runs the $\text{KeyGen}(1^\lambda, n, t, \ell)$ algorithm to obtain $pk, pkc, \{(sk_{1,0}^j, sk_{1,1}^j), \dots, (sk_{n,0}^j, sk_{n,1}^j)\}_{j \in [\ell]}$
3. Next, the challenger computes $(c^{(0)} = (c_0^{(0)}, c_1^{(0)}), k^{(0)}) \leftarrow \text{Enc}(pk, u)$, $(c^{(1)} = (c_0^{(1)}, c_1^{(1)}), k^{(1)}) \leftarrow \text{Enc}(pk, u)$, samples a bit $b \leftarrow \{0, 1\}$ and proceeds as follows:
4. If $b = 0$, the challenger sets $c^* = (c_0^{(0)}, c_1^{(1)})$ (in this case c^* is a mixture of $c^{(0)}$ and $c^{(1)}$).
5. If $b = 1$, the challenger sets $c^* = (c_0^{(d)}, c_1^{(d)})$ (in this case, c^* is equal to $c^{(0)}$ or $c^{(1)}$).
6. The challenger sends $pk, pkc, c^*, k^{(0)}$, $((sk_{1,0}^j, sk_{1,1}^j), \dots, (sk_{n,0}^j, sk_{n,1}^j))_{j \neq u}$ and $((sk_{1,d}^u, \dots, (sk_{n,d}^u))$
7. Finally, the adversary returns its guess $b' \in \{0, 1\}$. It wins this game if $b = b'$.

Fig. 3. OSS game for BTKEM $\mathcal{E}$ and adversary $\mathcal{A}$

thanks to an additional tracing algorithm. Hence, a traitor tracing threshold KEM is a threshold KEM as given in section 2.1 with the following additional procedure:

- a tracing algorithm named $\text{Trace}(pk, 1^{1/\varepsilon}, D)$ that takes as input the public key pk , an error bound ε , and a decoder D , and that outputs a subset $\mathcal{J} \subseteq [n]$.

The main difference with the one given in [5] is that we consider a public tracing algorithm, while theirs is private. Hence, we do not need a tracing key. It follows that the KeyGen is not modified compared to the one given in Section 2.1 (and the one in [5]), and only the public key is given on input to the tracing algorithm.

Decoder based security. In line with modern security models, we consider a stronger class of device called *distinguisher*. Instead of directly outputting a ciphertext, *distinguisher* first outputs two messages, then takes as input a ciphertext and outputs a bit to guess which plaintext is encrypted in the ciphertext. In a KEM, the device does not output the two plaintexts but instead takes as input a ciphertext and a session key and output a bit to guess whether the ciphertext is a valid encryption of the key or not.

This kind of device was introduced in [14] after finding then fixing a flaw in [6] with the classical definition of a pirate decoder.

The formal definition of a distinguisher is given as follow.

Definition 8. Let $\varepsilon = \varepsilon(\lambda)$ be a function of the security parameter and negl a negligible function of the security parameter $\lambda \in \mathbb{N}$. A traitor tracing threshold

KEM II = (KeyGen, Enc, Dec, Combine, Trace) with key space $\mathcal{K} = \{\mathcal{K}_\lambda\}$ is ε -secure if for every probabilistic polynomial time algorithm $\mathcal{A}$ and for every $\lambda \in \mathbb{N}$, the following holds:

$$\Pr[\text{GoodTrace}] \geq \Pr[\text{GoodDec}] - \text{negl}(\lambda) \quad \text{and} \quad \Pr[\text{BadTrace}] \leq \text{negl}(\lambda)$$

The events GoodTrace, GoodDec and BadTrace are defined as follows:

- Event GoodDec occurs when:

$$\Pr \left[D(c, k_b) = b : \begin{array}{l} (c, k_0) \leftarrow \text{Enc}(pk) \\ k_1 \leftarrow \mathcal{K} \\ b \leftarrow \{0, 1\} \end{array} \right] \geq \frac{1}{2} + \varepsilon.$$

- Event GoodTrace occurs when $\mathcal{J}' \neq \emptyset$ and $\mathcal{J}' \subseteq \mathcal{J}$, i.e. Trace identified users only in the alliance.
- Event BadTrace occurs when $\mathcal{J}' \neq \emptyset$ and $\mathcal{J}' \not\subseteq \mathcal{J}$, i.e. Trace accused a user that is not in the coalition of traitors.

Where $D, \mathcal{J}, \mathcal{J}'$ are output of the trace experiment in figure 4.

1. The adversary outputs a number of users n , a threshold t and sends them to the challenger.
2. The challenger runs the KeyGen algorithm to obtain the public key pk , the public combiner key pkc and the set of users secret keys $\{sk_1, \dots, sk_n\}$.
3. The adversary can request corruption queries on user $i \in [n]$. i is then added to the set of corrupted users $\mathcal{J}$. The challenger answers by sending i 's secret key sk_i .
4. After some time, the adversary outputs a distinguisher D and sends it to the challenger.
5. The challenger obtains $\mathcal{J}'$ by running the Trace algorithm using decoder D .

Fig. 4. Trace experiment TraceExperiment for a traitor tracing threshold KEM and adversary $\mathcal{A}$

We also define the following advantage:

$$\text{Adv}_{\mathcal{T}\mathcal{T}}^{TT}(\mathcal{A}) = \max\{\Pr[\text{GoodDec}] - \Pr[\text{GoodTrace}], \Pr[\text{BadTrace}]\}$$

2.4 DDH Assumption

We recall the classical formulation of the Decisional Diffie Hellman Hardness (DDH) assumption, that will be used for our construction of the q -partite KEM.

Definition 9. Let $\mathcal{G}$ be a group generation algorithm outputting a group $\mathbb{G}$ of prime order p and a generator g . We say that the Decisional Diffie-Hellman (DDH) problem is hard relative to $\mathcal{G}$ if for every probabilistic polynomial time algorithm $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$ and $x, y, z \leftarrow \mathbb{Z}_p$, we have that

$$\text{Adv}_{\mathcal{G}, \mathcal{A}}^{\text{DDH}}(\lambda) = |\Pr[\mathcal{A}(\mathbb{G}, p, g, g^x, g^y, g^{xy}) = 1] - \Pr[\mathcal{A}(\mathbb{G}, p, g, g^x, g^y, g^z) = 1]| \leq \text{negl}(\lambda)$$

2.5 Robust IPP Codes

Let $\mathcal{C} = \{\omega_1, \dots, \omega_n\} \subset \Sigma^\ell$ be a q -ary code of size n and length ℓ . Let Δ be the minimum Hamming distance over alphabet $\Sigma = \{1, \dots, q\}$. We assume that $\omega_i = \omega_{i,1} \dots \omega_{i,\ell} \in \Sigma^\ell$. Given a positive integer t , a subset of codewords $X = \{\omega_1, \dots, \omega_t\} \subset \mathcal{C}$ is called a coalition of size t . Let $X_i = \{\omega_{1,i}, \dots, \omega_{t,i}\}$ be the set of the i -th coordinate of the coalition X . If the cardinality of X_i is equal to 1, the coordinate i is called *undetectable*, else it is called *detectable*. The set of detectable coordinates for the coalition X is denoted by $D(X)$. The set of descendants of X , denoted $\text{desc}(X)$, is defined by $\text{desc}(X) = \{x = (x_1, \dots, x_\ell) \in \Sigma^\ell \mid x_j \in X_j, 1 \leq j \leq \ell\}$.

In the coalition X , we call codewords the parents of the set $\text{desc}(X)$. We define a t -descendant of the code $\mathcal{C}$ the set $\text{desc}_t(\mathcal{C}) = \bigcup_{X \subset \mathcal{C}, |X| \leq t} \text{desc}(X)$. The $\text{desc}_t(\mathcal{C})$ consists of all ℓ -tuples that could be generated by some coalition of size at most t . Codes with identifiable parent property (IPP) are then given by the following definition.

Definition 10. *Given a code $\mathcal{C} \subseteq \Sigma^\ell$, with parameters (ℓ, n, q) . Let $t \geq 2$ be an integer. The code $\mathcal{C}$ is called a t -IPP code if for all $x \in \text{desc}_t(\mathcal{C})$, it holds that*

$$\bigcap_{X \subset \mathcal{C}, |X| \leq t, \text{ s.t. } x \in \text{desc}(X)} X \neq \emptyset.$$

In a t -IPP code, we can always identify a parent codeword of any descendant $x \in \text{desc}_t(\mathcal{C})$. The *marking assumption*, introduced in [7] then states that any coalition of users can only create a codeword that lies in the set of descendants of the coalition.

We say that a code $\mathcal{C}$ is a *robust IPP* code if identification of the parents of a codeword x created by a coalition X is possible, even when x breaks the marking assumption in at most $\varepsilon\ell$ positions. We consider two types of mutations: erasure when $x_i = *$ and replacement when $x_i \in \Sigma \setminus X_i$. We say a word is ε -noisy when at most a fraction ε of its letter are $*$ mutations. Let $X \subset \mathcal{C}$, $|X| \leq t$ be a coalition. Assume $\text{desc}(X)$ is not empty. Let $x \in \text{desc}(X)$ such that x follows the marking assumption except in $\varepsilon\ell$ positions. Call i a coordinate of x a *mutation* if $x_i \notin X_i$. We denote by $\text{desc}(X)_\varepsilon$ the set of all vectors x formed from the vectors in the coalition X such that $x_i \in X_i$ for $\ell(1 - \varepsilon)$ coordinates i and x_i is a mutation in at most $\varepsilon\ell$ coordinates. Codes with robust identifiable parent property (RobustIPP) are formally defined as follows.

Definition 11. *Given a code $\mathcal{C} \subseteq \Sigma^\ell$ with parameters (ℓ, n, q) , let $t \geq 2$, be an integer. The code $\mathcal{C}$ is called a (t, ε) -IPP (robust t -IPP) if it holds that*

$$\bigcap_{X \subset \mathcal{C}, |X| \leq t, x \in \text{desc}(X)_\varepsilon} X \neq \emptyset$$

Informally speaking, the code $\mathcal{C}$ guarantees exact identification of at least one member of the pirate coalition of size at most t for any collusion attack with at

most $\varepsilon\ell$ mutations. A robust IPP code is said to have the *traceability property* if for any $x \in \text{desc}(X)$, the codeword $c \in \mathcal{C}$ closest to x by the Hamming distance is always one of the parents of x , i.e $c \in \bigcap_{X \subset \mathcal{C}, |X| \leq t, x \in \text{desc}(X)_\varepsilon} X$. This implies that a pirate can be identified by finding any vector $c \in \mathcal{C}$ such that its distance to x is the shortest. We shall use robust IPP with traceability property.

Requirement on Δ . From [12], the minimal distance Δ must verify:

$$\Delta > \ell \left(1 - \frac{1}{t^2} + \varepsilon \left(\frac{1}{t} - \frac{1}{t^2} \right) \right) = \ell \left(1 + \frac{\varepsilon}{t} - \frac{1 - \varepsilon}{t^2} \right)$$

3 Construction of the q Partite Threshold KEM

In this section, we give our first contribution by extending the notion of Bipartite Threshold KEM (BTKEM), introduced in [5], to q -Partite Threshold KEM.

3.1 Overview of the Construction

Extention from bipartite to q -partite. A q -Partite TKEM (QTKEM) scheme is a generalization of the Bipartite TKEM scheme. Users receive more keys, namely $q\ell$ keys instead of only 2ℓ and ciphertexts now have q sides instead of 2.

A QTKEM needs to have *all-sided correctness*, which is the direct generalization of the two sided correctness as well as what we call adaptive one-sided security. Loosely speaking, an adversary in possession of keys up to a certain letter $\alpha \in [q]$ should not be able to extract any meaningful information from a ciphertext using parts related to letters $\alpha + 1, \alpha + 2, \dots, q$.

After the parameters have been generated, this property lets the adversary choose on which position $j \in [\ell]$ and letter α he wishes to be tested on. The authority sends all the secret keys of all the users for position $j' \neq j$ and for position j he sends key $sk_{i,1}, \dots, sk_{i,\alpha-1}$, for all i . The adversary also receives a session key k and a ciphertext $c = (c_1, \dots, c_q)$ from the challenger and the adversary has to distinguish whether the α -th part of the ciphertext, c_α , is a proper encryption of the key k or a uniformly random element.

Note that in the one sided security game from [5], the adversary is required to choose the position and part he wishes to be tested upon *before* the challenger runs the key generation algorithm. This is in opposition with ours, where the adversary can adaptively choose the position and letter *after* the key generation.

3.2 Definition of q Partite KEM

We give the formal definition of a q -Partite Encryption KEM as follows:

Definition 12. A QTKEM $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Combine})$ is composed of the following algorithms:

- $\Pi.\text{KeyGen}(1^\lambda, n, t, \ell, q)$: Takes as input a security parameter λ , the number of users n , a threshold t , a number ℓ of positions and a number q an alphabet size. It outputs a public key pk and the set of users secret keys $(sk_1, \dots, sk_n)$, where for all $i \in [n]$, $sk_i = \{sk_{i,1}^j, \dots, sk_{i,q}^j\}_{j \in [\ell]}$.
- $\Pi.\text{Enc}(pk, j)$: Takes as input the public key pk and a position $j \in [\ell]$ and output a key k and a ciphertext $c = (c_1, \dots, c_q)$.
- $\Pi.\text{Dec}(j, sk_{i,\alpha}^j, c = (c_1, \dots, c_q))$: Takes a position $j \in [\ell]$, a user secret key $sk_{i,\alpha}^j$ with $i \in [n]$ and $\alpha \in [q]$ and outputs d_i . the decryption share of user i .
- $\Pi.\text{Combine}(pkc, c = (c_1, \dots, c_q), j, \mathcal{J}, \{d_i\}_{i \in \mathcal{J}})$: Takes as input the public combiner key pkc , a ciphertext c , a position $j \in [\ell]$, a subset $\mathcal{J} \subseteq [n]$ and a set of decryption shares $\{d_i\}_{i \in \mathcal{J}}$ and outputs either a key k or a rejection symbol $\perp$.

A QTKEM is required to satisfy the definitions of all-sided correctness and CPA security.

All-sided correctness. We need a re-visioned definition of the correctness on the QTKEM that we call *all-sided correctness*, that is the classical definition of correctness for a Threshold KEM should hold regardless of which ciphertext part was used by each user to obtain their share, as long as they have enough:

Definition 13 (Correctness). For all $n = n(\lambda)$, $t = t(\lambda)$, $\ell = \ell(\lambda)$, $q = q(\lambda)$, $\mathcal{J} \subseteq [n]$, s.t $|\mathcal{J}| = t$, a QTKEM scheme is said to be correct if

$$\text{Combine}(pkc, c, \mathcal{J}, j, \{d_i\}_{i \in \mathcal{J}}) = k,$$

where $\forall j \in [\ell]$, $(k, c = (c_1, \dots, c_q)) = \text{Enc}(pk, j)$ and $\forall i \in \mathcal{J}$, $\forall \alpha \in [q]$, $d_i = \text{Dec}(j, sk_{i,\alpha}^j, c = (c_1, \dots, c_q))$.

Semantic security. We require our QTKEM to be CPA secure. We adapt the definition given in section 2 to one that fits our new notion.

Additionally, the challenger samples a position j at random which is going to be used for the challenge. We denote by $\mathcal{K}$ the key space of the scheme that will be explicated in the construction. The security is presented in the security game in figure 5.

Definition 14. We say that Π is CPA secure if all probabilistic polynomial time algorithm $\mathcal{A}$ have at most negligible advantage in the game in figure 5, where $\mathcal{A}$'s advantage is defined as $\text{Adv}_\Pi^{\text{CPA}}(\mathcal{A}) = \left| \text{Succ}_\Pi^{\text{CPA}}(\mathcal{A}) - \frac{1}{2} \right|$ and $\text{Succ}_\Pi^{\text{CPA}}(\mathcal{A}) = \Pr[b = b']$ is the probability that the adversary outputs the correct bit in the game in figure 5.

1. The adversary outputs a number of parties n and a threshold t , a length ℓ and an alphabet size q , given a security parameter λ .
2. The challenger runs $\text{KeyGen}(1^\lambda, n, t, \ell, q)$ to generate $(pk, pkc, sk_1, \dots, sk_n)$.
3. At any time of the game, the adversary can corrupt party $i \in [n]$ and receives sk_i from the challenger. If the adversary corrupts more than t parties, the challenger stops the game.
4. The challenger samples a position $u \leftarrow [\ell]$, a bit $b \leftarrow \{0, 1\}$, a ciphertext and key pair $(k^{(0)}, c) \leftarrow \text{Enc}(pk, u)$ and a random key $k^{(1)} \leftarrow \mathcal{K}$. Finally, the challenger sends $k^{(b)}, c$ to the adversary.
5. The adversary can eventually corrupt more parties. The challenger stops the game if the total number of corrupted parties exceeds t .
6. Finally, the adversary returns its guess $b' \in \{0, 1\}$. It wins this game if $b = b'$.

Fig. 5. CPA security for QTKEM and adversary $\mathcal{A}$

Adaptive one-sided security. Eventually, we need another security requirement that we call adaptive one-sided security that is required for the tracing procedure to work in the traitor tracing scheme. There are two notable differences with the one-sided security from [5]. First, the adversary is allowed to adaptively choose the challenge position j and letter α . Second, the adversary receives all the secret keys of all the users, except for the selected position j where he receives all the secret keys except the α -th of each user. Then, he has to distinguish whether the α -th part of a the challenge ciphertext is an encryption of the challenge key or is random while the first $\alpha - 1$ parts are well-formed (i.e valid encryption of the key) and the $q - \alpha$ last parts are ill formed (i.e encryption of a another key sampled uniformly at random). It is made formal in the game presented in figure 6. In the original one-sided security, the adversary is required to distinguish between a mixture of a well formed ciphertext. In the former case, the two parts of the ciphertext are not encryption of the same key. In the latter, the two ciphertext are encryption of the same key which may or may not be the key sent along the challenge, depending on the adversary's choice.

Definition 15. *We say that a QTKEM Π is adaptively one-sided secure if all probabilistic polynomial time algorithm $\mathcal{A}$ have at most negligible advantage in the game in figure 6, where $\mathcal{A}$'s advantage is defined as $\text{Adv}_{\Pi}^{\text{ad-OSS}}(\mathcal{A}) = \left| \text{Succ}^{\text{ad-OSS}}(\mathcal{A}) - \frac{1}{2} \right|$ and $\text{Succ}^{\text{ad-OSS}}(\mathcal{A}) = \Pr[b = b']$ is the probability that the adversary outputs the correct bit in the game in figure 6.*

Note that for the tracing procedure to work, it is important to let the adversary choose the position j and letter α , since all of them will be tested. However, the challenger needs to know in advance the position and the letter for the challenge, because he modifies one specific public key, as it will be explicitly shown in the proof (see 4). It implies that in our security game, the challenger has to make a prediction on the choice of j and α made by the adversary by sampling in advance a j^* and a α^* and continues the game only if the adversary chooses

1. The adversary, given a security parameter λ , outputs a number of parties n , a threshold t , a number of positions ℓ and an integer q and sends them to the challenger.
2. The challenger runs the $\text{KeyGen}(1^\lambda, n, t, \ell, q)$ and outputs $(pk, pkc, \{(sk_{i,1}^j, \dots, sk_{i,q}^j)_{i \in [n]}\}_{j \in [\ell]})$ it sends pk, pkc to the adversary.
3. The adversary outputs a position j^* , a letter α^* and sends them to the challenger.
4. The challenger samples $b \in \{0, 1\}$, $(k^{(0)}, c^{(0)}) \leftarrow \text{Enc}(pk, j^*)$ and $(k^{(1)}, c^{(1)}) \leftarrow \text{Enc}(pk, j^*)$ and generates the challenge ciphertext $c^* = (c_1^{(0)}, \dots, c_{\alpha^*-1}^{(0)}, c_{\alpha^*}^{(b)}, c_{\alpha^*+1}^{(1)}, \dots, c_q^{(1)})$. Then, the challenger sends to the adversary $k^{(0)}, c^*, \{(sk_{i,1}^j, \dots, sk_{i,q}^j)_{i \in [n]}\}_{j \in [\ell] \setminus \{j^*\}}$ as well as $(sk_{i,1}^{j^*}, \dots, sk_{i,\alpha^*-1}^{j^*}, sk_{i,\alpha^*+1}^{j^*}, sk_{i,q}^{j^*})_{i \in [n]}$.
5. Finally, the adversary outputs its guess bit $b' \in \{0, 1\}$. The adversary wins this game if $b = b'$.

Fig. 6. *ad-OSS* for QTKEM and adversary $\mathcal{A}$

the same pair. Otherwise, the challenger stops the game. The probability of this happening is $\frac{1}{q\ell}$ if the position and letter are sampled uniformly over their respective set.

3.3 Construction

We now construct a q -Partite Threshold KEM $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Combine})$ from the Decisional Diffie Hellman assumption. We assume the existence of a generation algorithm $\mathcal{G}$ that takes as input a security parameter $\lambda \in \mathbb{N}$ and outputs a tuple $(\mathbb{G}, p, g)$ where $\mathbb{G}$ is a cyclic group of order p and g is a generator. Given a security parameter λ and a number of users n , we require that $1 \leq t \leq n$.

- $\Pi.\text{KeyGen}(1^\lambda, n, t, \ell, q)$: Takes as input the security parameter λ , the number of users n , the threshold t , a length ℓ and alphabet size q . It samples $(q+1)\ell$ uniformly random elements $x_j, y_{j,1}, \dots, y_{j,q}$ from $\mathbb{Z}_q$. It sets the public key as $pk = (pk_1, \dots, pk_\ell)$ where $pk_j = (X_j, Y_{j,1}, \dots, Y_{j,q})$ and $X_j = g^{x_j}$, $Y_{j,1} = g^{y_{j,1}}, \dots, Y_{j,q} = g^{y_{j,q}}$. Then, let $s_{j,1}, \dots, s_{j,n}$ be the shares obtained from the (t, n) -Shamir Secret Sharing of x_j . It sets the secret keys as $sk_{i,\alpha}^j = \frac{s_{j,i}}{y_{j,\alpha}}$, for $j \in [\ell], i \in [n], \alpha \in [q]$.
- $\Pi.\text{Enc}(pk, j)$: Takes as input the public key pk and a position j . To obtain a session key, it chooses uniformly at random r in $\mathbb{Z}_q$ and sets the session key as $k = X_j^r$. Then, it produces a ciphertext with q parts $c = (c_1, \dots, c_q)$ where $c_\alpha = Y_{j,\alpha}^r$ for $\alpha \in [q]$. The algorithm outputs the session key and the ciphertext (k, c) .
- $\Pi.\text{Dec}(j, sk_{i,\alpha}^j, c)$: Takes as input a position j , one of the position's associated secret keys $sk_{i,\alpha}^j$ of user i and a ciphertext c . The algorithm outputs the decryption share $d_i = c_\alpha^{sk_{i,\alpha}^j}$.

- $\Pi.\text{Combine}(pkc, c, \mathcal{J}, \{d_i\}_{i \in \mathcal{J}})$: Takes as input the public key combiner pkc , a position j , a ciphertext c , a set of user $\mathcal{J}$ and the associated set of decryption shares $\{d_i\}_{i \in \mathcal{J}}$. If $|\mathcal{J}| < t$, the algorithm outputs an error symbol $\perp$. Otherwise, it computes the Lagrange polynomial interpolation coefficients $(\mu_i)_{i \in \mathcal{J}}$ and outputs the session key $k = \prod_{i \in \mathcal{J}} d_i^{\mu_i}$.

We require the QTKEM to have all-sided correctness. We now show how this property holds in our scheme.

Theorem 1. *Π scheme has all-sided correctness.*

Proof. Let $0 < t \leq n$, $\ell > 0$, $\mathcal{J} \subseteq [n]$ s.t $|\mathcal{J}| = t$. We have, for all $j \in [\ell]$, $\alpha \in [q]^n$, :

$$\begin{aligned}
\prod_{i \in \mathcal{J}} d_i^{\mu_i} &= \prod_{i \in \mathcal{J}} \prod_{\alpha_i=1}^q d_i^{\mu_i} \\
&= \prod_{i \in \mathcal{J}} \prod_{\alpha_i=1}^q (Y_{j, \alpha_i}^r)^{\mu_i \frac{s_{j,i}}{y_{j, \alpha_i}}} \\
&= \prod_{i \in \mathcal{J}} \prod_{\alpha_i=1}^q g^{ry_{j, \alpha_i} \mu_i \frac{s_{j,i}}{y_{j, \alpha_i}}} \\
&= \prod_{i \in \mathcal{J}} g^{rs_{j,i} \mu_i} \\
&= g^{r \sum_{i \in \mathcal{J}} \mu_i s_{j,i}} \\
&= g^{rx_j} = X_j^r = k
\end{aligned}$$

Note that α being a random element of $[q]^n$, this holds for every subset $\mathcal{J}$ of size t and is independent of the letters used.

We now prove the QTKEM to be semantically secure as well as adaptive one-sided secure under the standard DDH assumption. These properties are formalized in theorems 2, 3.

In this game, the challenger $\mathcal{B}$ chooses the position $u \in [\ell]$ of the challenge. For positions $j \neq u$, $\mathcal{B}$ behaves "honestly" in the sense that he samples the public and private keys normally. For position u , he sets the first component of the DDH challenge as the public key X_u , the second component as the ciphertext parts and the last component of the triplet as the session key k . If the DDH challenge is random, then so will be the session key, hence $\mathcal{B}$ is playing against $\mathcal{A}$ the CPA game with bit $b = 1$. In the other case, the challenge ciphertext is a valid encryption of the key hence $\mathcal{B}$ is playing the game with $b = 0$. Since $\mathcal{A}$ has a non negligible advantage in winning the CPA game, $\mathcal{B}$ forwards its guess.

Theorem 2. *Under the DDH hardness problem, Π is CPA-secure.*

Proof. Suppose there exists an adversary $\mathcal{A}$ that breaks the CPA property of the scheme. We build a simulator $\mathcal{B}$ that breaks the DDH assumption.

- $\mathcal{B}$ requests for a DDH tuple $(\mathbb{G}, p, g, A, B, C)$ with $A = g^a, B = g^b$ and $C = g^{ab}$ or $C = g^c$ for some $a, b, c \leftarrow \mathbb{Z}_p$. $\mathcal{B}$ invokes $\mathcal{A}(1^\lambda)$ and receives n, t, ℓ, q .
- $\mathcal{B}$ samples a position $u \leftarrow [\ell]$.
- $\mathcal{B}$ samples the public keys as follows:
 $x_1, \dots, x_{u-1}, x_{u+1}, \dots, x_\ell \leftarrow \mathbb{Z}_p, y_{1,1}, \dots, y_{1,q} \leftarrow \mathbb{Z}_p, \dots, y_{\ell,1}, \dots, y_{\ell,q} \leftarrow \mathbb{Z}_p$, then sets the public keys: $X_u = A, \forall j \in [\ell] \setminus \{u\}, X_j = g^{x_j}$ and $\forall \alpha \in [q], Y_{j,\alpha} = g^{y_{j,\alpha}}$, then $pk_j = (X_j, Y_{j,1}, \dots, Y_{j,q})$ and sends $pk = (pk_1, \dots, pk_\ell)$.
- Without loss of generality, assume $\mathcal{A}$ does $Q \leq t - 1$ different corruption queries without repeating itself. If it makes more then the challenger would interrupt the game. $\mathcal{B}$ samples $s_1, \dots, s_{t-1} \leftarrow \mathbb{Z}_p$ and answers to the ρ -th corruption queries of $\mathcal{A}$, corresponding to party $i \in [n]$ as follows.
 - For position $j \neq u$, $\mathcal{B}$ computes the secret keys honestly: samples $s_{j,1}, \dots, s_{j,n}$ the points of a (t, n) -Shamir Secret Sharing of x_j then sets $sk_{i,\alpha}^j = \frac{s_{j,i}}{y_{j,\alpha}}$ for $\alpha \in [q]$.
 - for position u , $\mathcal{B}$ sets $sk_{i,\alpha}^u = \frac{s_\rho}{y_{u,\alpha}}$

Then $\mathcal{B}$ replies to $\mathcal{A}$ with $\{(sk_{i,1}^j, \dots, sk_{i,\alpha}^j, \dots, sk_{i,q}^j)_{i \in [n]}\}_{j \in [\ell]}$. The keys are distributed honestly even for position u since $\mathcal{A}$ queries at most $Q \leq t - 1$ parties so x_u cannot be recovered and the keys are uniformly distributed over $\mathbb{Z}_p^Q$.

Next, $\mathcal{B}$ sets the session key as $k = C$, the ciphertext parts $c_1 = B^{y_{u,1}}, \dots, c_q = B^{y_{u,q}}$, the challenge ciphertext as $c = (c_1, \dots, c_q)$ and sends (k, c) to $\mathcal{A}$.

$\mathcal{B}$ perfectly simulates the CPA security game to $\mathcal{A}$. If $C = g^{ab}$ then it simulates the game with $b = 0$ and if $C = g^c$ then it simulates the game with $b = 1$. It follows that:

$$\text{Adv}_{\mathcal{A}, \mathcal{G}}^{\text{ddh}}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{CPA}}(\lambda)$$

■

In theorem 3, we build a simulator $\mathcal{B}$ that receives a DDH triplet and uses an adversary against the *adaptive one-sided security* property. The challenger makes a guess on the pair (j, α) that will be chosen by the adversary. He then sets the α -th part of the ciphertext as the third component of the DDH challenge, the public key $Y_{j,\alpha}$ as the first component of the challenge and the session key k as the second. If the DDH challenge is a random group element, then the key and the α -th part of the challenge will not be related, hence $\mathcal{B}$ is playing against $\mathcal{A}$ the *adaptive one-sided security* game with bit $b = 1$. On the other hand, if the challenge is not random, $\mathcal{B}$ is playing the game with bit $b = 0$. Since $\mathcal{A}$ has a non negligible advantage in winning this game, $\mathcal{B}$ forwards the guess.

Note that this happens only if $\mathcal{B}$ had guessed correctly the pair (j, α) made by $\mathcal{A}$, which happens with probability $1/q\ell$. This is necessary since $\mathcal{B}$ needs to set the public keys before knowing the choice of $\mathcal{A}$. This point differs from [5] since the adversary $\mathcal{A}$ must choose the position $j \in [\ell]$ and a side $d \in \{0, 1\}$ before the challenger runs the setup.

Theorem 3. *Under the DDH hardness assumption, Π is adaptive one-sided secure.*

Proof. Suppose the existence of an adversary $\mathcal{A}$ that can break the adaptive one-sided security. We build a simulator $\mathcal{B}$ that breaks the DDH assumption.

- $\mathcal{B}$ requests for a DDH tuple $(\mathcal{G}, p, g, X, Y, Z)$ with $X = g^x, Y = g^y$ for some uniformly random $x, y \leftarrow \mathbb{Z}_p$ and $Z = g^{xy}$ or $Z = g^z$. $\mathcal{B}$ invokes $\mathcal{A}(1^\lambda)$ and receives n, t, ℓ, q .
- $\mathcal{B}$ samples $\alpha^* \leftarrow [q]$ and $j^* \leftarrow [\ell]$, and samples the keys as follows:
- For position $j \neq j^*$ $\mathcal{B}$ samples the keys honestly: it computes $X_j = g^{x_j}, Y_{j,\alpha} = g^{y_{j,\alpha}}$ for $x_j, y_{j,1}, \dots, y_{j,q} \leftarrow \mathbb{Z}_q$. $\mathcal{B}$ samples $s_{j,1}, \dots, s_{j,n}$ the shares of a (t, n) -Shamir Secret Sharing of x_j . Finally $\mathcal{B}$ computes $pk_j = (X_j, Y_{j,1}, \dots, Y_{j,q})$ and for $i \in [n], \alpha \in [q], sk_{i,\alpha}^j = \frac{s_{j,i}}{y_{j,\alpha}}$.
- For position j^* , $\mathcal{B}$ samples $x_{j^*}, y_{j^*,1}, \dots, y_{j^*,\alpha^*-1}, y_{j^*,\alpha^*+1}, \dots, y_{j^*,q} \leftarrow \mathbb{Z}_q$, and computes $X_{j^*} = g^{x_{j^*}}, Y_{j^*,\alpha} = g^{y_{j^*,\alpha}}, \alpha \neq \alpha^*$, samples $s_{j^*,1}, \dots, s_{j^*,n}$ the points of a (t, n) -Shamir Secret Sharing of x_{j^*} . Then it sets $Y_{j^*,\alpha^*} = X_{j^*}, pk_{j^*} = (X_{j^*}, Y_{j^*,1}, \dots, Y_{j^*,q})$ and $\forall i \in [n], \forall \alpha \in [q] \setminus \{\alpha^*\}, sk_{i,\alpha}^{j^*} = \frac{s_{j^*,i}}{y_{j^*,\alpha}}$. $\mathcal{B}$ finally sets $pk = (pk_1, \dots, pk_n)$ and $pkc = \perp$ and sends them to $\mathcal{A}$.
- $\mathcal{B}$ requests $\mathcal{A}$ and receives a position j and a letter α . If $(j^*, \alpha^*) \neq (j, \alpha)$, $\mathcal{B}$ stops the game and restarts it from the beginning.
- Then, $\mathcal{B}$ sets $\forall \alpha \in [\alpha^* - 1], c_\alpha = Y^{y_{j^*,\alpha}}, c_{\alpha^*} = Z, \forall \alpha \in [\alpha^* + 1, q], c_\alpha \leftarrow \mathbb{G}, k^{(0)} = Y^{x_{j^*}}$ and $c = (c_1, \dots, c_q)$.

$\mathcal{B}$ sends $k^{(0)}, \{(sk_{i,1}^j, \dots, sk_{i,q}^j)_{i \in [n]}\}_{j \in [\ell] \setminus \{j^*\}}, (sk_{i,1}^{j^*}, \dots, sk_{i,\alpha^*-1}^{j^*}, sk_{i,\alpha^*+1}^{j^*}, \dots, sk_{i,q}^{j^*})_{i \in [n]}$ to $\mathcal{A}$, who replies with a guess bit b' and $\mathcal{B}$ outputs the same bit b' .

If $Z = g^{xy}$, $\mathcal{B}$ perfectly simulates the *ad-OSS* game with $b = 0$. Otherwise if $Z = g^z$, Z is a uniformly random element of $\mathbb{G}$ independent of c^* and $k^{(0)}$. In this case, it perfectly simulates one-sided security game with $b = 1$. It follows that:

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \mathcal{G}}^{\text{ddh}}(\lambda) &= |\Pr[\mathcal{A}(\mathbb{G}, p, g, g^x, g^y, g^{xy}) = 1] - \Pr[\mathcal{A}(\mathbb{G}, p, g, g^x, g^y, g^z) = 1]| \\ &= \frac{1}{2} |\Pr[b' = 1 \cap (j^*, \alpha^*) = (j, \alpha) \mid b = 1] \\ &\quad - \Pr[b' = 1 \cap (j^*, \alpha^*) = (j, \alpha) \mid b = 0]| \end{aligned}$$

Given that the probability that the challenger guesses correctly the choice of the adversary for position j and letter α is $1/q\ell$, we then have

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \mathcal{G}}^{\text{ddh}}(\lambda) &= \frac{1}{q\ell} |\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]| \\ &= \frac{1}{2q\ell} |\Pr[b' = 1 \mid b = 1] - (1 - \Pr[b' = 0 \mid b = 0])| \\ &= \frac{1}{2q\ell} |2\Pr[b' = b] - 1| \\ &= \frac{1}{2q\ell} \text{Adv}^{\text{ad-OSS}}(\mathcal{A}) \end{aligned}$$

■

4 A Traitor Tracing for Threshold Decryption scheme

In this section, we use the q -Partite KEM alongside a robust IPP code to construct a Traitor Tracing scheme. Thanks to the code properties, our scheme benefits from the desired public traceability property. Note that since the coalition of traitors we consider is of size $f \geq t$, it is large enough to decrypt on its own. Thus, we can use classical tools such as linear tracing used for robust IPP codes. Hence, our tracing strategy is inspired from classical traitor tracing such as [12].

4.1 Construction

Let n be the maximal number of users, $t \leq n$ the threshold value and ε the advantage of a pirate distinguisher. Each party is identified by an integer from $\{1, \dots, n\}$. Let $\mathcal{C}$ be an IPP code with parameters (ℓ, n, q) , able to identify up to coalitions of size f . Let Δ be its Hamming distance and $0 < \theta = \frac{1/2-\varepsilon}{1/2-2/\sqrt{\lambda}} < \frac{1}{f+1}$ satisfying the condition:

$$\Delta > \ell \left(1 + \frac{\theta}{f} - \frac{1-\theta}{f^2} \right)$$

Next, let $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Combine})$ be a q -Partite Threshold KEM. Finally, given a security parameter λ , we assume the size of the traitors verifies $t \leq f \leq n$. We define a traitor tracing scheme $\mathcal{TT}$ composed of the tuple of probabilistic polynomial time algorithms $\mathcal{TT} = (\text{KeyGen}, \text{Enc}, \text{Dec}, \text{Combine}, \text{Trace})$ as follows:

$\mathcal{TT}.\text{KeyGen}(1^\lambda, n, t, 1^\varepsilon)$: Takes as input the security parameter λ , the number of users n , the threshold t and the decoder advantage ε . By calling the $\Pi.\text{KeyGen}(1^\lambda, n, t, \ell, q)$, we obtain the public key pk , the public key combiner pkc and the set of secret keys $\{(sk_{i,1}^j, \dots, sk_{i,\alpha}^j, \dots, sk_{i,q}^j)_{i \in [n]}\}_{j \in [\ell]}$. Finally, for each user i , set: $sk_i = (sk_{i,\omega_{i,1}}^1, \dots, sk_{i,\omega_{i,\ell}}^\ell)$ where $\omega_{i,j}$ is the value at position j of codeword ω_i .

$\mathcal{TT}.\text{Enc}(pk)$: Takes as input the public key pk . Samples a position $j \in [\ell]$. The ciphertext is $c = (j, c' = (c_1, \dots, c_q))$ and the session key is k , where k and c' were obtained using the encryption function $\Pi.\text{Enc}(pk, j)$.

$\mathcal{TT}.\text{Dec}(sk_i, c)$: Takes as input the public key pk , the position j , a user secret key sk_i , and a ciphertext $c = (j, c')$. By calling the decryption function $\Pi.\text{Dec}(j, sk_{i,\omega_{i,j}}^j, c')$ on key $sk_{i,\omega_{i,j}}^j$ from sk_i , the user gets his decryption share d_i .

$\mathcal{TT}.\text{Combine}(pkc, c, \mathcal{J}, \{d_i\}_{i \in \mathcal{J}})$: Takes as input the public parameters pk , the public key combiner pkc , a ciphertext $c = (j, c')$, a set of t users $\mathcal{J}$ with the associated set of decryption shares d_i . By calling the function $\Pi.\text{Combine}(pkc, j, c', \mathcal{J}, \{d_i\}_{i \in \mathcal{J}})$

, the user recovers the session key k .

$\mathcal{TT}.\text{Trace}(pk, D, 1^\varepsilon)$: Takes as input the public key pk , a decoder D and an error bound ε . We run the decoder for every position $j \in [\ell]$ to form new letters of the pirate codeword ω^* . After all the positions have been tested, we use the Trace algorithm from $\mathcal{C}$ to accuse a party. We consider the following tracing procedure:

```

for  $j = 1$  to  $\ell$  do
   $count \leftarrow 0$ 
  for  $\alpha = 0$  to  $q + 1$  do
    loop  $W \leftarrow 8\lambda^{3/2}(n/\varepsilon)^2$  times
       $(k^{(0)}, c^{(0)}) = (c_1^{(0)}, \dots, c_q^{(0)}) \leftarrow \Pi.\text{Enc}(pk, j)$ 
       $(k^{(1)}, c^{(1)}) = (c_1^{(1)}, \dots, c_q^{(1)}) \leftarrow \Pi.\text{Enc}(pk, j)$ 
       $b \leftarrow \{0, 1\}$ 
       $c^* \leftarrow (j, c_1^{(b)}, \dots, c_{\alpha-1}^{(b)}, c_\alpha^{(1-b)}, \dots, c_q^{(1-b)})$ 
      Call  $D$  on input  $(c^*, k^{(b)})$ 
      if  $D(c^*, k^{(b)}) = b$  then
         $count \leftarrow count + 1$ 
      end if
    end loop
  Let  $p_{j,\alpha} \leftarrow count/W$  be the fraction of time where  $D$  outputs  $b$  correctly.
  end for
  If there is a letter  $\alpha^* \in [1, q]$  such that  $|p_{j,\alpha^*+1} - p_{j,\alpha^*}| \geq \frac{\varepsilon}{q}$  then letter  $\alpha^*$ 
  is accused and  $\omega_j \leftarrow \alpha^*$ .
  Else,  $\omega_j$  is assigned *.
end for

```

From the pirate word $\omega = \omega_1 \dots \omega_\ell$ found after the loop finished, call tracing procedure in $\mathcal{C}$ on input ω . The Trace returns a traitor.

4.2 Security of the Tracing

The correctness and semantic security of the scheme are immediate by the all-sided correctness and semantic security of the underlying q -Partite KEM. The Trace procedure only relies on the public key and an access to the pirate distinguisher D . Likewise, $\mathcal{C}$'s tracing procedure can be used with public information only. Hence, our scheme has public traceability.

Security of Tracing. The goal of the tracing procedure is to form a pirate word ω and use the trace algorithm of code $\mathcal{C}$ to identify a user. The algorithm creates the word letter by letter using the QTKEM and modifies part after part the ciphertext it sends to the distinguisher. On iteration $\alpha = 0$, the ciphertext produced by the algorithm is completely well formed, i.e all the ciphertext parts are valid encryption of the session key. Hence, the probability for the distinguisher

to successfully distinguish is maximum since it has at least one key for each position.

On each iteration, starting from the end, the tracing procedure then switches a new part of the ciphertext, from a well-formed encryption of the session key to a random group element.

On the final iteration $\alpha = q+1$, all the ciphertext parts have been replaced by random group elements, unrelated to the session key. Therefore, the distinguisher has a negligible probability of successfully distinguishing.

If a significant gap in the probabilities of success is noticed by the tracing procedure, i.e., if the probability of success drops when the α -th part of the ciphertext is switched from a correctly formed to a random element, then letter α is accused.

By the IPP code's property, a user will be correctly accused as long as a the proportion of letter that deviates from the marking assumptions is smaller than θ .

Theorem 4. *For every probabilistic polynomial time algorithm $\mathcal{A}$ against the ϵ -security of the scheme $\mathcal{TT}$, there exists a probabilistic polynomial time algorithm $\mathcal{B}$ against the One Sided Security of the q -Partite Threshold KEM Π such that for all $\lambda \in \mathbb{N}$, the following holds:*

$$\text{Adv}_{\mathcal{A}, \mathcal{TT}}^{TT} \leq 4\ell e^{-4\lambda \frac{n^2}{q\epsilon^2}} + \frac{2\ell q^3}{q-f} \text{Adv}^{ad-OSS}(\mathcal{B})$$

where $\ell = \ell(\lambda)$, $q = q(\lambda)$ are the codewords length and alphabet size as defined in the scheme and f is the size of the coalition.

By property of the IPP code $\mathcal{C}$, when given a codeword ω as input to its tracing procedure, the code outputs one of the parent codewords with probability one as long as the proportion of letters that deviates from the marking assumption (def 11) does not exceed θ .

We prove the theorem in two steps. First, lemma 1 shows that if **GoodDec** occurs, then the proportion of $*$ symbols in the codeword does not exceeds θ . Then lemma 2 shows that whenever a letter is accused then it must be that the letter is in the coalition or otherwise it would be possible to construct a reduction algorithm that wins the *ad-OSS* security property with non negligible advantage.

Put together, lemmas 1 and 2 prove that $\Pr[\text{BadTrace}] = 0$ and that $\Pr[\text{GoodDec}] - \Pr[\text{GoodTrace}]$ is negligible whenever the codeword formed at the end of the Trace algorithm deviates less than $\theta\ell$ times from the marking assumption.

Call a position $j \in [\ell]$ *good* if:

$$\Pr \left[D((j, c), k_b) = b : \begin{array}{l} (c, k_0) \leftarrow \Pi.\text{Enc}(pk, j) \\ k_1 \leftarrow \mathcal{K}, \quad b \leftarrow \{0, 1\} \end{array} \right] \geq \frac{1}{2} + \frac{2}{\sqrt{\lambda}}. \quad (1)$$

Loosely speaking, even when event **GoodDec** occurs, the decoder box may never work for some position j . Lemma 1 shows that if there was too few *good*

positions then event **GoodDec** could not occur, and bounds the probability to set ω_j to $*$ i.e if a position is *good* then a letter will be accused. Lemma 2 then bounds the probability that the accused letter is not in the coalition.

Lemma 1. *Let $\varepsilon = \varepsilon(\lambda)$ be a non-negligible function of the security parameter $\lambda \in \mathbb{N}$. If event **GoodDec** occurs, then it holds that:*

$$\Pr[\omega^* \text{ is not } \theta\text{-noisy}] \leq 4e^{-4\lambda \frac{n^2}{q\varepsilon^2}}$$

.

Proof. Suppose on the contrary that there is a proportion at least θ of $*$ symbols in the pirate codeword, then we have that:

$$\begin{aligned} \Pr \left[D(c, k_b) = b : \begin{array}{l} (c, k_0) \leftarrow \mathcal{TT}.\text{Enc}(pk) \\ k_1 \leftarrow \mathcal{K}, b \leftarrow \{0, 1\} \end{array} \right] &< (1 - \theta)1 + \theta \left(\frac{1}{2} + \frac{2}{\sqrt{\lambda}} \right) \\ &= 1 + \left(\frac{2}{\sqrt{\lambda}} - \frac{1}{2} \right) \theta \\ &= 1 + \frac{\frac{1}{2} - \varepsilon}{\frac{1}{2} - \frac{2}{\sqrt{\lambda}}} \left(\frac{2}{\sqrt{\lambda}} - \frac{1}{2} \right) \\ &= \frac{1}{2} + \varepsilon. \end{aligned}$$

This is in contradiction with the hypothesis that **GoodDec** occurs.

Next, we prove that for every *good* position, the probability for the j -th letter of ω being $*$ is negligible.

Let $j \in [\ell]$ be a good position. We have

$$\left| \Pr \left[D((j, c_1^{(0)}, \dots, c_q^{(0)}), k^{(1)}) = 1 \right] - \Pr \left[D((j, c_1^{(1)}, \dots, c_q^{(1)}), k^{(1)}) = 1 \right] \right| \geq \frac{4}{\sqrt{\lambda}} \quad (2)$$

where for $b \in \{0, 1\}$, $c = (c_1^{(b)}, \dots, c_q^{(b)}) \leftarrow \mathcal{II}.\text{Enc}(pk, j)$. By the triangle inequality, there is at least one letter $\alpha \in [q]$ such that the following equations holds:

$$\begin{aligned} & \left| \Pr \left[D((j, c_1^{(1)}, \dots, c_{\alpha-1}^{(1)}, c_{\alpha}^{(0)}, c_{\alpha+1}^{(0)}, \dots, c_q^{(0)}), k^{(1)}) = 1 \right] - \right. \\ & \quad \left. \Pr \left[D((j, c_1^{(1)}, \dots, c_{\alpha}^{(1)}, c_{\alpha+1}^{(0)}, \dots, c_q^{(0)}), k^{(1)}) = 1 \right] \right| \geq \frac{2}{q\sqrt{\lambda}} \quad (3) \end{aligned}$$

Let us consider the set of random variables X_{α} that increases by one each time the decoder outputs 1 on input $((c_1^{(1)}, \dots, c_{\alpha-1}^{(1)}, c_{\alpha}^{(0)}, \dots, c_q^{(0)}), k^{(1)})$. We consider that X_0 (resp X_q) is defined using the decoder on input $((c_1^{(0)}, \dots, c_q^{(0)}), k^{(1)})$

(resp $((c_1^{(1)}, \dots, c_q^{(1)}), k^{(1)})$) Without loss of generality, suppose that:

$$\Pr\left[D((j, c_1^{(1)}, \dots, c_{\alpha-1}^{(1)}, c_{\alpha}^{(0)}, \dots, c_q^{(0)}), k^{(1)}) = 1\right] > \Pr\left[D((j, c_1^{(1)}, \dots, c_{\alpha}^{(1)}, c_{\alpha+1}^{(0)}, \dots, c_q^{(0)}), k^{(1)}) = 1\right]$$

This means that X_{α} is distributed as a binomial random variable with W trials and some probability parameter $\xi \in [0, 1]$, and $X_{\alpha} + 1$ is distributed as a binomial random variable with W trials and probability parameter $\xi' \geq \xi + \frac{2}{q\sqrt{\lambda}}$.

Since letter $\omega_j \leftarrow \alpha$ when $|p_{j,\alpha+1} - p_{j,\alpha}| \geq \frac{\varepsilon}{q}$, which is realised whenever X_{α} and $X_{\alpha+1}$ are both at most $\frac{W}{2q\sqrt{\lambda}}$ far away from their expected values. It follows that :

$$\begin{aligned} \Pr[\omega_j \leftarrow \alpha] &= \Pr[|X_{\alpha+1} - X_{\alpha}| \leq W\varepsilon/q] \\ &\leq \Pr\left[\left(X_{\alpha+1} < \mathbb{E}(X_{\alpha-1}) - \frac{W}{2q\sqrt{\lambda}}\right) \cup \left(X_{\alpha} < \mathbb{E}(X_{\alpha}) + \frac{W}{2q\sqrt{\lambda}}\right)\right] \\ &\leq 2\Pr_{X \leftarrow \text{Bin}(W, 1/2)}\left[|X - \mathbb{E}| > \frac{W}{2q\sqrt{\lambda}}\right] \end{aligned} \quad (4)$$

Here, X is a random variable sampled from the Binomial distribution with parameters W and $1/2$. Eq. 4 follows from convexity. Finally, by the Chernoff Bound, we get that:

$$\Pr[\omega_j \leftarrow \alpha] \leq 4e^{-4\lambda \frac{n^2}{q\varepsilon^2}} \quad (5)$$

Note that if the traitor's letter is 1 (resp q), we do the same analysis but we consider the random variable X_0 (resp X_{q+1}) obtained by using the decoder on pair $((c_1^{(0)}, \dots, c_q^{(0)}), k^{(1)})$ (resp $((c_1^{(1)}, \dots, c_q^{(1)}), k^{(1)})$). For each position j , there must be at least some letter $\alpha \in [q]$ that satisfies the condition imposed by Eq. 3. Hence, the probability that the Trace algorithm sets $\omega \leftarrow *$ is at least $1 - 4e^{-4\lambda \frac{n^2}{q\varepsilon^2}}$. The lemma follows from taking the union bound on all good positions, with at most ℓ of them. ■

Now, we show that the probability to accuse an innocent user is negligible. If the created codeword ω^* does not belong to the set of descendant codewords $\text{desc}(\mathcal{J})$, then it must be that, for some *good* position $j^* \in [\ell]$, the letter $\omega_{j^*}^*$ is inconsistent with every j^* -th letter of the coalition, i.e. $\omega_{j^*}^* \notin \mathcal{J}_{j^*}$. This means that there is a good position j^* and a letter α^* for which $|p_{j^*, \alpha^*+1} - p_{j^*, \alpha^*}| \geq \frac{\varepsilon}{q}$, when running the decoder D . If such a position and letter exist, then we show that we can construct a reduction algorithm that wins the *ad-OSS* game of the underlying QTKEM Π with non negligible probability, by feeding the decoder with the challenger ciphertext c^* . Depending on the choice of ciphertext in the

ad-OSS game, the challenge c^* is distributed either as a ciphertext of the form $(c_1^{(0)}, \dots, c_{\alpha^*}^{(0)}, c_{\alpha^*+1}^{(1)}, \dots, c_q^{(1)})$ that we use on D to compute p_{j^*, α^*+1} or as a ciphertext of the form $(c_1^{(0)}, \dots, c_{\alpha^*-1}^{(0)}, c_{\alpha^*}^{(1)}, \dots, c_q^{(1)})$ that we use on D to compute p_{j^*, α^*} . The result follows from the fact that the two distributions are at least ε/q far apart.

Lemma 2. *Assuming the QKTEM II is ad-OSS secure then the probability to accuse an innocent user is negligible. Formally, suppose there is an adversary $\mathcal{A}$ capable of producing a decoder D such that: there is a good position j^* and a letter α^* for which $|p_{j^*, \alpha^*+1} - p_{j^*, \alpha^*}| \geq \frac{\varepsilon}{q}$. Then, one can construct an algorithm $\mathcal{B}$ such that:*

$$\text{Adv}_{II}^{\text{ad-OSS}}(\mathcal{B}) \geq \frac{(q-f)\varepsilon}{2\ell q^3}$$

Proof. We construct a reduction algorithm $\mathcal{B}$ that wins the *ad-OSS* game with non-negligible advantage. The reduction algorithm $\mathcal{B}$ starts by requesting parameters n, t to $\mathcal{A}$ and sends n, t, ℓ, q to the *ad-OSS* challenger, which responds by sending the public key pk and public combiner key pkc . Next, $\mathcal{B}$ sends $j \leftarrow [\ell], \alpha \leftarrow [q]$ and receives $(k, c^* = (c_1^*, \dots, c_q^*)), \{sk_{i, \alpha'}^{j'}\}_{i \in [n], \alpha' \in [q], j' \in [\ell] \setminus \{j\}}$ as well as $\{sk_{i, \alpha'}^{j'}\}_{i \in [n], \alpha' \neq \alpha}$. Then, $\mathcal{B}$ samples an IPP code $\mathcal{C}$ with parameters (ℓ, n, q) , and answers corruption queries of $\mathcal{A}$. When $\mathcal{A}$ requests for secret key of party i , $\mathcal{B}$ answers by sending $\omega_i, sk_i = \{sk_{i, \omega_{i, j'}}^{j'}\}$. If $\mathcal{A}$ asks for corruption of a party for which $\mathcal{B}$ does not know the entire secret key, i.e. a user i such that $\omega_{i, j} = \alpha$, $\mathcal{B}$ stops the game and outputs a random bit b' . Denote by **Stop** this event. In the worst case, $\mathcal{A}$ makes one query for each member of the coalition and all their j -th letter are different (otherwise it would only decrease the probability of the event **Stop** to happen). Then we have that $\Pr[\text{Stop}] \leq f/q$. We condition the rest of the analysis on the fact that the event **Stop** does not happen (hence $\mathcal{B}$ does not output b' randomly). Next, $\mathcal{B}$ uses the decoder on input k, c^* and let b' be the output of the decoder. $\mathcal{B}$ simply forwards its guess to the challenger. In case the bit chosen by the *ad-OSS* challenger is $b = 0$, the c^* is of the form $(c_1^{(0)}, \dots, c_{\alpha-1}^{(0)}, c_{\alpha}^{(0)}, c_{\alpha+1}^{(1)}, \dots, c_q^{(1)})$ distributed as the ciphertext given to D to compute $p_{j, \alpha+1}$. On the other hand, if the bit chosen by the challenger is $b = 1$, c^* is of the form $(c_1^{(0)}, \dots, c_{\alpha-1}^{(0)}, c_{\alpha}^{(1)}, c_{\alpha+1}^{(1)}, \dots, c_q^{(1)})$ and is distributed as the ciphertext given to D to compute $p_{j, \alpha}$. It follows that (recall that we condition the analysis on $\neg \text{Stop}$, that j is the *good* position α the letter such that $|p_{j, \alpha+1} - p_{j, \alpha}| \geq \frac{\varepsilon}{q}$, i.e. $j = j^*$ and $\alpha = \alpha^*$):

$$\begin{aligned}
\Pr[b' = b] &= \frac{1}{2} (\Pr[b' = 0 \mid b = 0] + \Pr[b' = 1 \mid b = 1]) \\
&= \frac{1}{2} (\Pr[b' = 0 \mid b = 0] - \Pr[b' = 0 \mid b = 1]) + \frac{1}{2} \\
&= \frac{1}{2} \left(\Pr[b' = 0 \mid c^* = (c_1^{*(0)}, \dots, c_{\alpha^*}^{*(0)}, c_{\alpha^*+1}^{*(1)}, \dots, c_q^{*(1)})] \right. \\
&\quad \left. - \Pr[b' = 0 \mid c^* = (c_1^{*(0)}, \dots, c_{\alpha^*-1}^{*(0)}, c_{\alpha^*}^{*(1)}, \dots, c_q^{*(1)})] \right) + \frac{1}{2} \\
&= \frac{1}{2} (p_{j^*, \alpha^*+1} - p_{j^*, \alpha^*}) + \frac{1}{2}
\end{aligned}$$

Since α^* is the letter being accused, we have $|p_{j^*, \alpha^*+1} - p_{j^*, \alpha^*}| \geq \varepsilon/q$. Since $\mathcal{B}$ chooses the correct *good* position j^* and letter α^* with probability $1/q\ell$ and that event $\overline{\text{Stop}}$ happens with probability at least $(q-f)/q$, it follows that the advantage in the previous game of the reduction algorithm is at least:

$$\text{Adv}_{\Pi}^{\text{ad-OSS}}(\mathcal{B}) \geq \frac{(q-f)\varepsilon}{2\ell q^3}$$

By combining lemma 1 with lemma 2, the theorem follows. ■

5 Perspectives

While this paper has been focusing on the decentralization of the tracing procedure for a threshold decryption scheme, an orthogonal line of work in broadcast encryption focuses on the revocation of users. When a scheme combines both the revocation and the tracing property, it is called Trace and Revoke. This has already been studied in the literature [20,19,17,13,8,1], [21]. However none of them yet extends to the threshold setting. In this section, we discuss how we can modify the q -partite KEM scheme to achieve both tracing and revocation. The resulting solution, though inefficient, will not be fully detailed here. Instead, we provide a sketch to illustrate the possibility of such a construction and to motivate further research in this direction.

In the q -partite encryption scheme, switching a part of the ciphertext into a random group element amounts to revoking the decryption capability of the associated letter. Next, by extending the length of the ciphertext, i.e., encrypting not only for a single position but several of them, it becomes possible to have a fine grain control on which letter can be revoked and in which position. A ciphertext would take the following form : $c = (c_1, \dots, c_\ell)$ where $c_j = (c_{j,1}, \dots, c_{j,q})$ is a QTKEM encryption for position j . Hence, when user receives only a portion of the secret keys, as it is the case when they are identified by a codeword, switching parts of the ciphertext into random elements in different positions is equivalent to revoking the decryption capability of a codeword and therefore revokes the user identified by this codeword. Lastly, the q -partite scheme must be

adapted to take into account a new requirement: a user must be able to obtain his/her decryption share *no matter which part and which position* was used during the decryption. This can be done easily by modifying the KeyGen algorithm. Instead of sampling x_j for each $j \in [\ell]$, sample only one single x and split it into n shares $s_1, \dots, s_n$. This allows a user to obtain the same decryption share no matter which position and part of the ciphertext he/she used. In order to keep the tracing property, we can rely a similar technique used in [12] where the user is required to use several of his/her secret keys (since he/she has one key per position). In order to do so, each share s_i is then secret shared again into ℓ shares $s_{i,1}, \dots, s_{i,\ell}$ using a (κ, ℓ) Secret Sharing scheme (it does not necessarily have to be Shamir's Secret Sharing). The shares $s_{i,j}$ are the ones used to set the secret keys. This forces a user to run the decryption procedure on several positions in order to obtain his/her decryption share. κ must be chosen such that the code's property ensures that there is at least κ positions which differ in the codewords of legitimate users and revoked users.

The shortcoming of this technique, very similar to the one in [12], is the increase of the length of the ciphertexts. The growth is in $O(q\ell)$ where ℓ is very large, making the scheme extremely inefficient.

We thus leave as an open problem the construction of an efficient trace and revoke scheme for threshold decryption. Finally, anonymity in broadcasting, which is a strong functionality that implies both tracing and revocation, remains a topic that has not yet been addressed in threshold decryption. For a threshold broadcast scheme to be anonymous, the shares would also need to be anonymous. Otherwise, an adversary possessing several shares could identify their creators by checking their validity, thereby breaking the classical definition of anonymity. This challenge requires a novel approach to tackle this problem.

Finally, [5] introduces a second construction called BTBF which is a BTKEM with constant public key size. Since, this is out of the primary focus of this paper - public traceability, we leave it as an open problem to provide a construction and a rigorous security analysis that our QTKEM can be adapted to reduce the size of the public keys in a similar fashion as [5].

Acknowledgments. We would like to thank anonymous reviewers for their helpful discussions and valuable comments. We would like to thank Orel Cosseron for his valuable insights during the revision of the paper. This work was supported by the France 2030 ANR Projects ANR-22-PECY-003 SecureCompute and the Banque Publique d'Investissement under the VisioConfiance project. This work is licensed under a "CC BY 4.0" license.

References

1. Agrawal, S., Bhattacharjee, S., Phan, D.H., Stehlé, D., Yamada, S.: Efficient public trace and revoke from standard assumptions: Extended abstract. In: Thuringham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) ACM CCS 2017. pp. 2277–2293. ACM Press (Oct / Nov 2017). <https://doi.org/10.1145/3133956.3134041>

2. Barg, A., Kabatiansky, G.: Robust parent-identifying codes and combinatorial arrays. *IEEE Trans. Inf. Theor.* **59**(2), 994–1003 (feb 2013). <https://doi.org/10.1109/TIT.2012.2220522>, <https://doi.org/10.1109/TIT.2012.2220522>
3. Boneh, D., Franklin, M.K.: An efficient public key traitor tracing scheme. In: Wiener, M.J. (ed.) *CRYPTO'99*. LNCS, vol. 1666, pp. 338–353. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_22
4. Boneh, D., Kiayias, A., Montgomery, H.W.: Robust fingerprinting codes: a near optimal construction. In: *Proceedings of the Tenth Annual ACM Workshop on Digital Rights Management*. p. 3–12. DRM '10, Association for Computing Machinery, New York, NY, USA (2010). <https://doi.org/10.1145/1866870.1866873>, <https://doi.org/10.1145/1866870.1866873>
5. Boneh, D., Partap, A., Rotem, L.: Accountability for misbehavior in threshold decryption via threshold traitor tracing. In: Reyzin, L., Stebila, D. (eds.) *CRYPTO 2024, Part VII*. LNCS, vol. 14926, pp. 317–351. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68394-7_11
6. Boneh, D., Sahai, A., Waters, B.: Fully collusion resistant traitor tracing with short ciphertexts and private keys. In: Vaudenay, S. (ed.) *EUROCRYPT 2006*. LNCS, vol. 4004, pp. 573–592. Springer, Berlin, Heidelberg (May / Jun 2006). https://doi.org/10.1007/11761679_34
7. Boneh, D., Shaw, J.: Collusion-secure fingerprinting for digital data (extended abstract). In: Coppersmith, D. (ed.) *CRYPTO'95*. LNCS, vol. 963, pp. 452–465. Springer, Berlin, Heidelberg (Aug 1995). https://doi.org/10.1007/3-540-44750-4_36
8. Boneh, D., Waters, B.: A fully collusion resistant broadcast, trace, and revoke system. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) *ACM CCS 2006*. pp. 211–220. ACM Press (Oct / Nov 2006). <https://doi.org/10.1145/1180405.1180432>
9. Boneh, D., Zhandry, M.: Multiparty key exchange, efficient traitor tracing, and more from indistinguishability obfuscation. In: Garay, J.A., Gennaro, R. (eds.) *CRYPTO 2014, Part I*. LNCS, vol. 8616, pp. 480–499. Springer, Berlin, Heidelberg (Aug 2014). https://doi.org/10.1007/978-3-662-44371-2_27
10. Chabanne, H., Phan, D.H., Pointcheval, D.: Public traceability in traitor tracing schemes. In: Cramer, R. (ed.) *EUROCRYPT 2005*. LNCS, vol. 3494, pp. 542–558. Springer, Berlin, Heidelberg (May 2005). https://doi.org/10.1007/11426639_32
11. Chor, B., Fiat, A., Naor, M.: Tracing traitors. In: Desmedt, Y. (ed.) *CRYPTO'94*. LNCS, vol. 839, pp. 257–270. Springer, Berlin, Heidelberg (Aug 1994). https://doi.org/10.1007/3-540-48658-5_25
12. Do, X.T., Phan, D.H., Yung, M.: A concise bounded anonymous broadcast yielding combinatorial trace-and-revoke schemes. In: Conti, M., Zhou, J., Casalicchio, E., Spognardi, A. (eds.) *ACNS 20International Conference on Applied Cryptography and Network Security, Part II*. LNCS, vol. 12147, pp. 145–164. Springer, Cham (Oct 2020). https://doi.org/10.1007/978-3-030-57878-7_8
13. Dodis, Y., Fazio, N.: Public key trace and revoke scheme secure against adaptive chosen ciphertext attack. In: Desmedt, Y. (ed.) *PKC 2003*. LNCS, vol. 2567, pp. 100–115. Springer, Berlin, Heidelberg (Jan 2003). https://doi.org/10.1007/3-540-36288-6_8
14. Goyal, R., Koppula, V., Waters, B.: Collusion resistant traitor tracing from learning with errors. In: Diakonikolas, I., Kempe, D., Henzinger, M. (eds.) *50th ACM STOC*. pp. 660–670. ACM Press (Jun 2018). <https://doi.org/10.1145/3188745.3188844>

15. Kiayias, A., Yung, M.: Breaking and Repairing Asymmetric Public-Key Traitor Tracing. In: Digital Rights Management—DRM '02. LNCS, vol. 2696, pp. 32–50. Springer (2002)
16. Kiayias, A., Yung, M.: On crafty pirates and foxy tracers. In: Sander, T. (ed.) Security and Privacy in Digital Rights Management. pp. 22–39. Springer Berlin Heidelberg, Berlin, Heidelberg (2002)
17. Kim, C.H., Hwang, Y.H., Lee, P.J.: An efficient public key trace and revoke scheme secure against adaptive chosen ciphertext attack. In: Lai, C.S. (ed.) ASIACRYPT 2003. LNCS, vol. 2894, pp. 359–373. Springer, Berlin, Heidelberg (Nov / Dec 2003). https://doi.org/10.1007/978-3-540-40061-5_23
18. Komaki, H., Watanabe, Y., Hanaoka, G., Imai, H.: Efficient asymmetric self-enforcement scheme with public traceability. In: Kim, K. (ed.) PKC 2001. LNCS, vol. 1992, pp. 225–239. Springer, Berlin, Heidelberg (Feb 2001). https://doi.org/10.1007/3-540-44586-2_17
19. Naor, D., Naor, M., Lotspiech, J.: Revocation and tracing schemes for stateless receivers. In: Kilian, J. (ed.) CRYPTO 2001. LNCS, vol. 2139, pp. 41–62. Springer, Berlin, Heidelberg (Aug 2001). https://doi.org/10.1007/3-540-44647-8_3
20. Naor, M., Pinkas, B.: Efficient trace and revoke schemes. In: Frankel, Y. (ed.) FC 2000. LNCS, vol. 1962, pp. 1–20. Springer, Berlin, Heidelberg (Feb 2001). https://doi.org/10.1007/3-540-45472-1_1
21. Nishimaki, R., Wichs, D., Zhandry, M.: Anonymous traitor tracing: How to embed arbitrary information in a key. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 388–419. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49896-5_14
22. Pfitzmann, B.: Trials of traced traitors. In: Information Hiding. LNCS, vol. 1174, pp. 49–64. Springer (1996)
23. Pfitzmann, B., Waidner, M.: Asymmetric fingerprinting for larger collusions. In: Graveman, R., Janson, P.A., Neuman, C., Gong, L. (eds.) ACM CCS 97. pp. 151–160. ACM Press (Apr 1997). <https://doi.org/10.1145/266420.266453>
24. Phan, D., Safavi-Naini, R., Tonien, D.: Generic construction of hybrid public key traitor tracing with full-public-traceability. In: Bugliesi, M., Preneel, B., Sassone, V., Wegener, I. (eds.) ICALP 2006, Part II. LNCS, vol. 4052, pp. 264–275. Springer, Berlin, Heidelberg (Jul 2006). https://doi.org/10.1007/11787006_23
25. Watanabe, Y., Hanaoka, G., Imai, H.: Efficient asymmetric public-key traitor tracing without trusted agents. In: Naccache, D. (ed.) CT-RSA 2001. LNCS, vol. 2020, pp. 392–407. Springer, Berlin, Heidelberg (Apr 2001). https://doi.org/10.1007/3-540-45353-9_29